

Web Application Security Basics — DVWA Lab Report

Environment: Kali Linux + DVWA running locally at **127.0.0.1:8081**. This report documents authorized testing only inside the intentionally vulnerable DVWA training lab.

1. Objective

Identify basic web vulnerabilities using DVWA, inspect HTTP traffic with Burp Suite, record evidence, and document practical mitigation techniques.

2. Tools and Environment

DVWA: deliberately vulnerable local web application.

Burp Suite Community: intercepts and inspects HTTP requests.

Kali Linux: security-testing operating system.

Browser: used to access DVWA.

3. Start and Verify DVWA

Open the local training application in the browser:

`http://127.0.0.1:8081`

The supplied evidence shows the DVWA login page and the DVWA Security page. The security level is set to **Low**, which is appropriate for this controlled learning exercise.

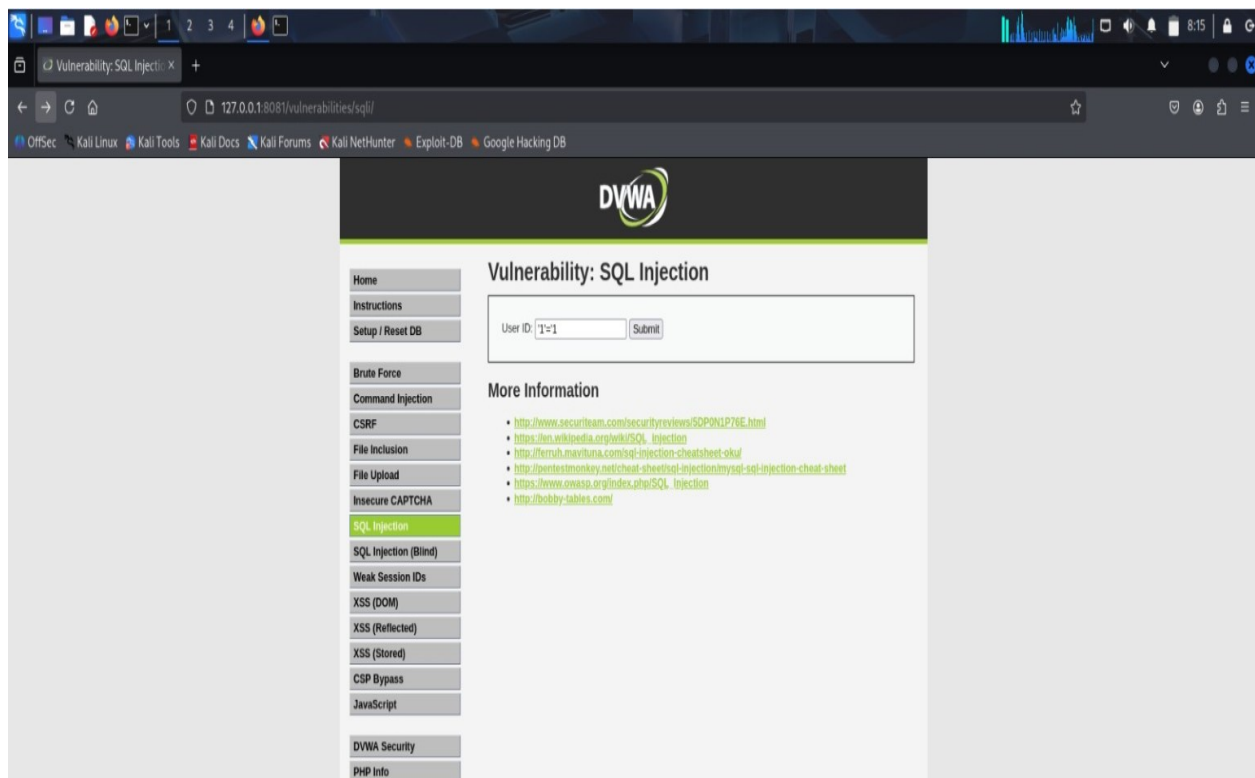


Figure 1. DVWA login page running locally.

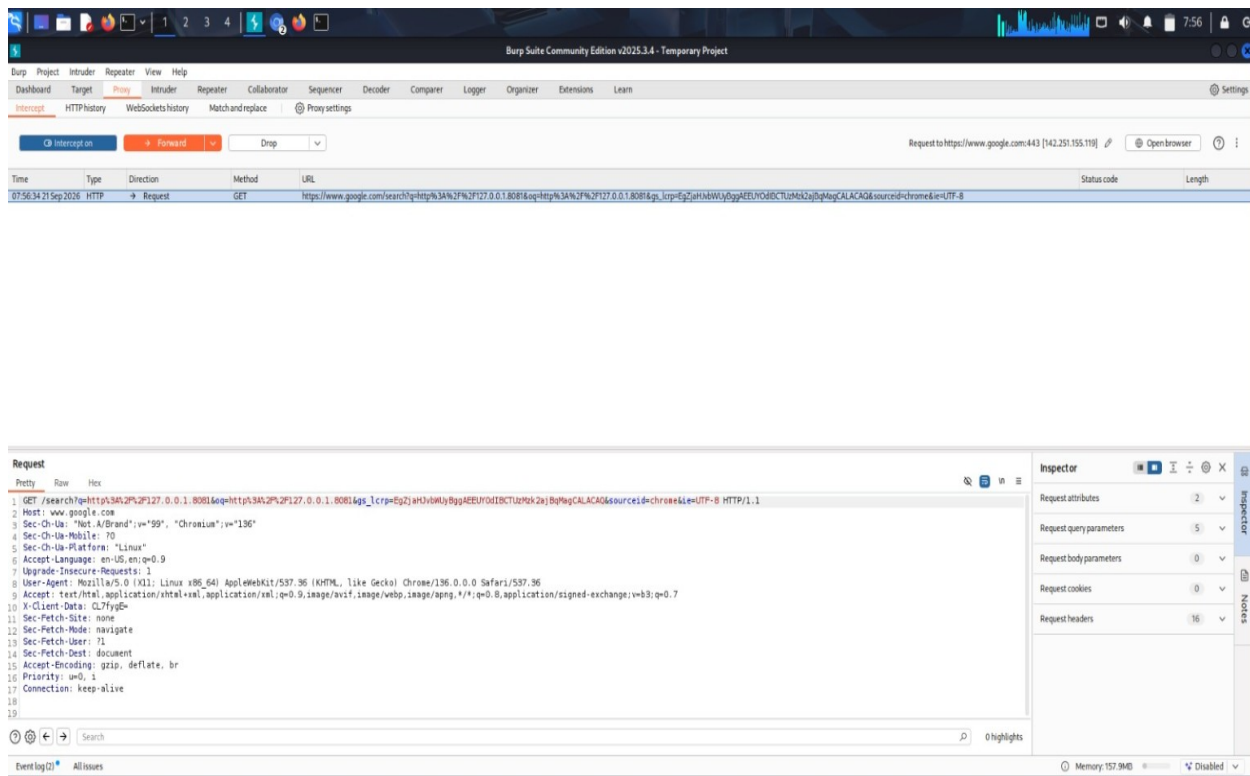


Figure 2. DVWA Security page showing Low security.

4. Burp Suite Request Interception

Open Burp Suite → Temporary Project → Start Burp. Select Proxy → Intercept and enable interception. Browse to DVWA and generate a request. Burp pauses the request so the tester can inspect the HTTP method, URL, headers, cookies and parameters before forwarding it.

Example:
 GET /vulnerabilities/xss_r/?name=TEST HTTP/1.1
 Host: 127.0.0.1:8081

The supplied Burp screenshot demonstrates interception. For final evidence, a request whose Host is 127.0.0.1:8081 is preferable to external browser traffic.

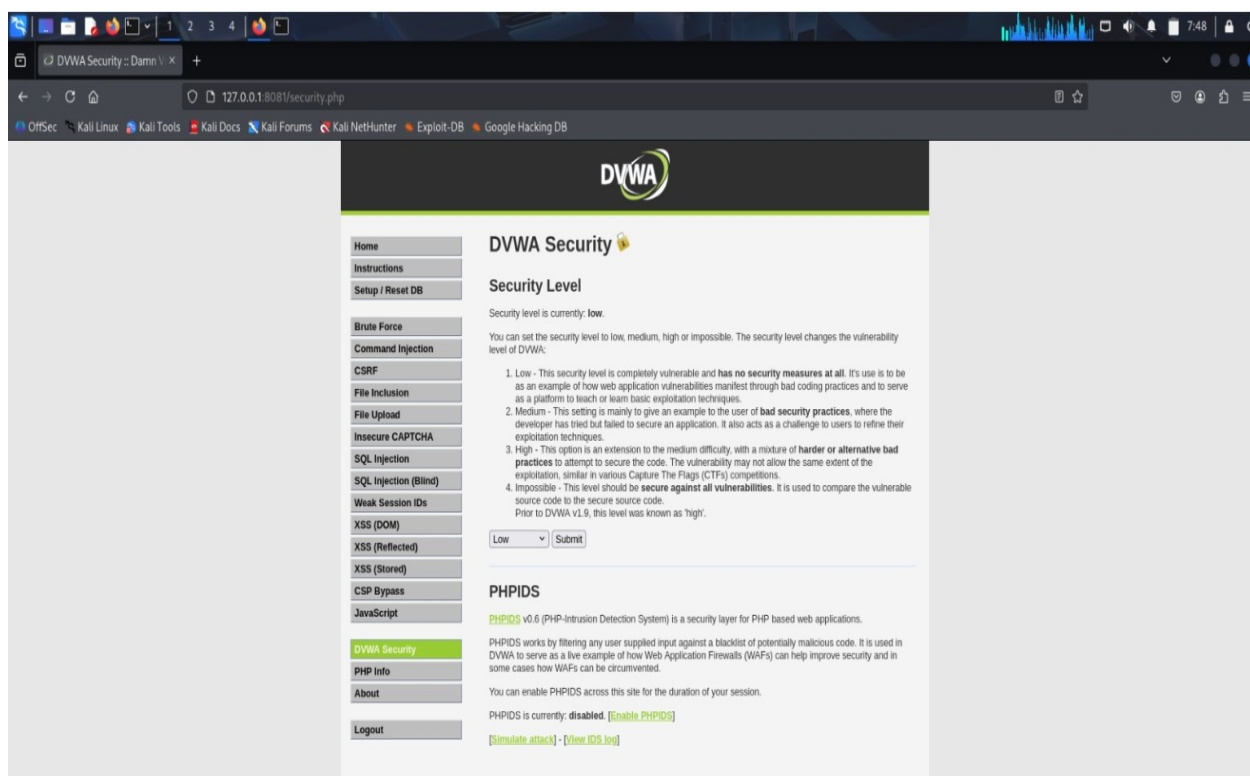


Figure 3. Burp Suite interception evidence supplied for the lab.

5. Reflected XSS

DVWA path: XSS (Reflected). Enter this harmless proof-of-concept into the name field:

```
<script>alert('XSS-Test')</script>
```

The supplied screenshot shows the payload being reflected by the DVWA page and represented in the URL. In this intentionally vulnerable lab, the test demonstrates why untrusted input must not be inserted into an executable browser context without appropriate output encoding.

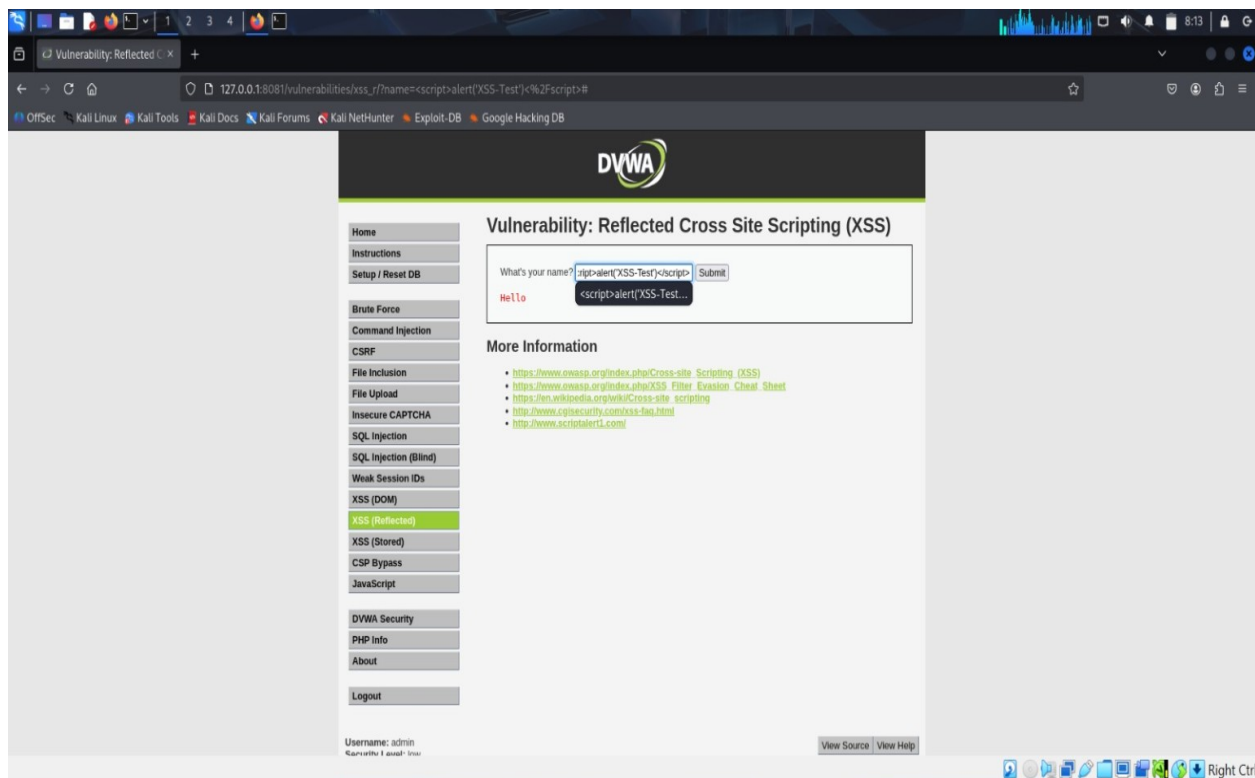


Figure 4. Reflected XSS test in DVWA.

Finding: Reflected Cross-Site Scripting (XSS).

Potential impact: In a real application, attacker-controlled script could execute in a victim's browser and affect page content, user actions or sensitive browser-side data, depending on context.

Mitigation: Context-aware output encoding, appropriate input validation, safe DOM/HTML APIs, and a restrictive Content Security Policy as defense in depth.

6. SQL Injection

DVWA path: SQL Injection. Use this controlled Boolean proof-of-concept:

```
1' OR '1'='1
```

In a deliberately vulnerable application, this input can alter SQL query logic and cause additional records to be returned. The supplied screenshot shows the SQL Injection page with the test input visible.

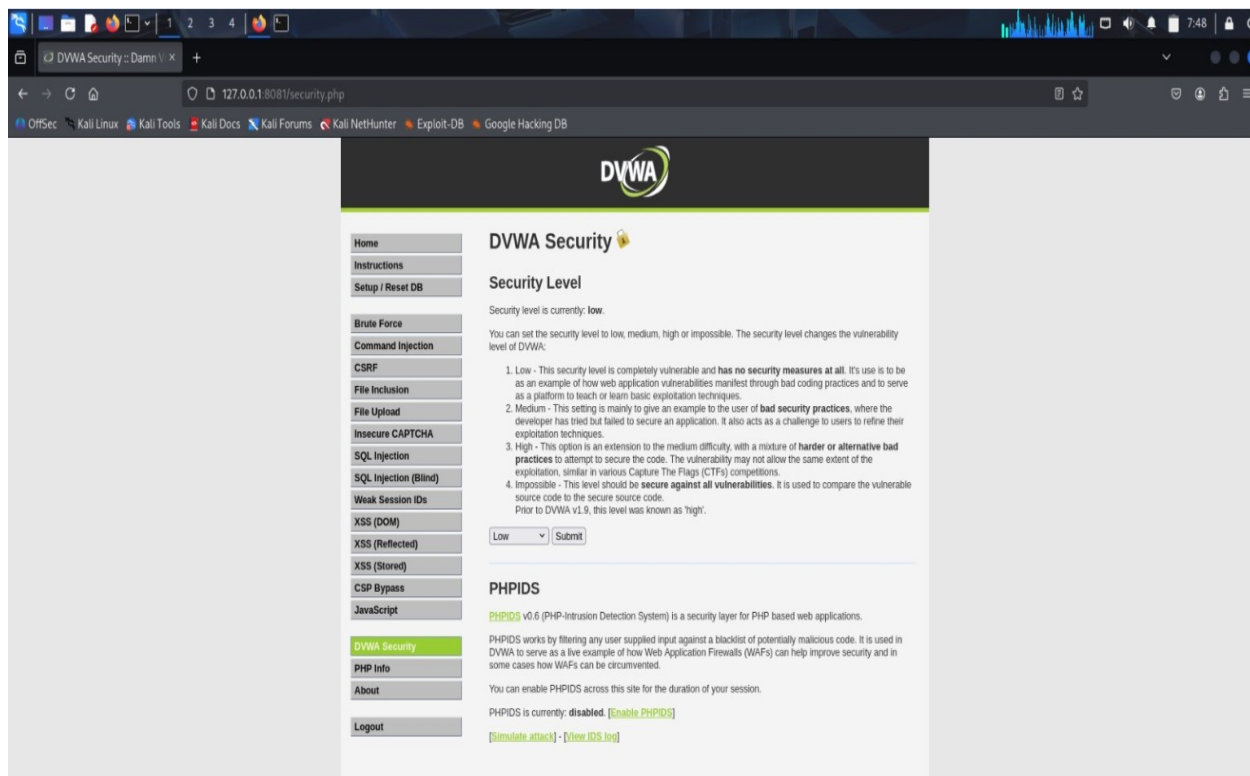


Figure 5. SQL Injection page and controlled test input.

Finding: SQL Injection demonstrated in the intentionally vulnerable DVWA environment.

Potential impact: In real applications, SQL Injection can permit unauthorized reading or modification of database data and may lead to further compromise depending on configuration.

Mitigation: Use prepared statements/parameterized queries, never concatenate untrusted input into SQL, validate input, and apply least-privilege database permissions.

7. Supporting Docker Evidence

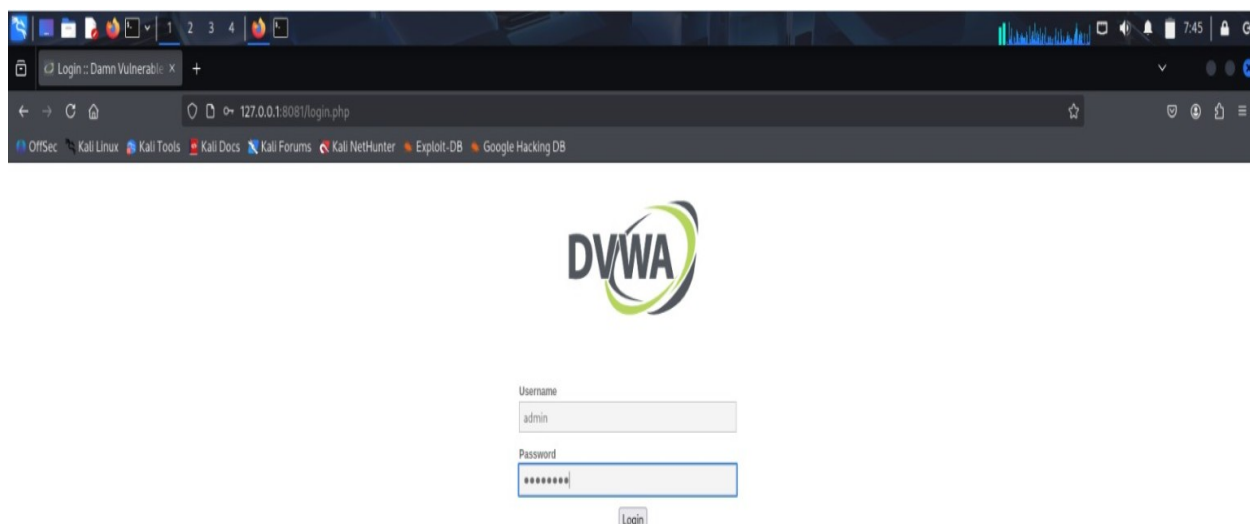


Figure 6. Docker terminal showing the DVWA image being downloaded/loaded.

The terminal evidence shows the DVWA image being loaded successfully. Network connectivity was also verified during setup.

8. Reproducible Lab Workflow

1. Start the local DVWA lab.
2. Open `http://127.0.0.1:8081`.
3. Confirm DVWA is reachable and log in.
4. Set DVWA Security to Low for this exercise.
5. Start Burp Suite.
6. Enable Proxy → Intercept.
7. Browse to a DVWA function and capture the request.
8. Test Reflected XSS with the controlled payload.
9. Test SQL Injection with the controlled Boolean payload.
10. Take screenshots of the evidence.
11. Record finding, impact and mitigation.
12. Stop/reset the lab when finished.

9. Mitigation Cheat Sheet

Issue	Primary mitigation
Reflected XSS	Context-aware output encoding; safe HTML/DOM APIs; input validation; CSP defense in depth.
SQL Injection	Prepared statements/parameterized queries; input validation; least-privilege database account.
HTTP testing	Use an authorized intercepting proxy to understand requests and responses; never test systems without permission.

10. What You Learned

- Burp Suite can intercept and inspect HTTP traffic.
- XSS occurs when untrusted input reaches an executable browser context without correct handling.
- SQL Injection occurs when untrusted input can change SQL query logic.
- A useful security report contains evidence, finding, impact and remediation.

11. Deliverables Checklist

- [✓] DVWA running locally
- [✓] Security level documented
- [✓] Burp interception procedure documented
- [✓] Reflected XSS test documented
- [✓] SQL Injection test documented
- [✓] Screenshots included
- [✓] Impact explained
- [✓] Mitigations explained
- [✓] Reproducible test inputs included

12. Conclusion

This exercise demonstrates a complete beginner web-security assessment workflow in a controlled DVWA environment: establish the lab, inspect HTTP traffic, perform safe proof-of-concept tests, collect evidence, explain the risk, and recommend remediation. The same structure can be reused for future authorized security labs and internship documentation.

Safety note: The payloads and procedures are intended for the local DVWA training environment or other systems for which you have explicit authorization.